LEZIONE 14 - Array Multidimensionali e Stringhe

PARTE TEORICA (2h)

1. Array Bidimensionali (Matrici)

Un **array bidimensionale** (o **matrice**) è un array di array: una struttura che organizza i dati in **righe** e **colonne**.

Rappresentazione concettuale:

```
Array bidimensionale: int matrice[3][4]

      Colonna:  0    1    2    3
               ┌────┬────┬────┬────┐
    Riga 0     │  1 │  2 │  3 │  4 │
               ├────┼────┼────┼────┤
    Riga 1     │  5 │  6 │  7 │  8 │
               ├────┼────┼────┼────┤
    Riga 2     │  9 │ 10 │ 11 │ 12 │
               └────┴────┴────┴────┘

matrice[0][0] = 1    matrice[0][1] = 2
matrice[1][2] = 7    matrice[2][3] = 12
```

Utilizzi comuni: - Tabelle di dati (es: vendite per mese/prodotto) - Immagini bitmap - Giochi da tavolo (scacchiera, tris) - Matrici matematiche - Griglie geografiche

---

2. Dichiarazione e Inizializzazione delle Matrici

Sintassi base:

```
tipo nomeMatrice[righe][colonne];
```

Esempi di dichiarazione:

```c
int matrice[3][4];        // 3 righe, 4 colonne (12 elementi totali)
float temperature[12][7]; // 12 mesi, 7 giorni/settimana
char scacchiera[8][8];    // Scacchiera 8×8
```

Inizializzazione:

```c
// Metodo 1: Inizializzazione completa
int matrice[3][4] = {
    {1, 2, 3, 4},      // Riga 0
    {5, 6, 7, 8},      // Riga 1
    {9, 10, 11, 12}    // Riga 2
};

// Metodo 2: Inizializzazione lineare (stesso risultato)
int matrice[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};

// Metodo 3: Inizializzazione parziale
int matrice[3][4] = {
    {1, 2},         // Riga 0: {1, 2, 0, 0}
    {5},            // Riga 1: {5, 0, 0, 0}
    {}              // Riga 2: {0, 0, 0, 0}
};

// Metodo 4: Inizializzazione a zero
int matrice[3][4] = {0};  // Tutti gli elementi a 0

// Metodo 5: Prima dimensione automatica
int matrice[][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};  // Il compilatore deduce 3 righe
```

Esempio completo:

```c
#include <stdio.h>

int main() {
    // Matrice 3×3 (identità)
    int identita[3][3] = {
        {1, 0, 0},
        {0, 1, 0},
        {0, 0, 1}
    };

    // Matrice non inizializzata
    float valori[2][3];

    // Inizializzazione manuale
    valori[0][0] = 1.5;
    valori[0][1] = 2.3;
    valori[0][2] = 3.7;
    valori[1][0] = 4.2;
    valori[1][1] = 5.8;
    valori[1][2] = 6.1;

    return 0;
}
```

---

3. Accesso agli Elementi

Sintassi:

```
nomeMatrice[indiceRiga][indiceColonna]
```

Accesso in lettura e scrittura:

```c
int matrice[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

// Lettura
int valore = matrice[1][2];  // valore = 7

// Scrittura
matrice[0][0] = 100;         // Primo elemento = 100
matrice[2][3] = 200;         // Ultimo elemento = 200

// Uso in espressioni
int somma = matrice[0][1] + matrice[1][1];  // 2 + 6 = 8
matrice[1][0] = matrice[0][0] * 2;          // matrice[1][0] = 200
```

Iterazione con cicli annidati:

```c
int matrice[3][4];
int righe = 3, colonne = 4;

// Riempimento
for (int i = 0; i < righe; i++) {
    for (int j = 0; j < colonne; j++) {
        matrice[i][j] = i * colonne + j + 1;
    }
}

// Stampa
printf("Matrice:\n");
for (int i = 0; i < righe; i++) {
    for (int j = 0; j < colonne; j++) {
        printf("%3d ", matrice[i][j]);
    }
    printf("\n");
}
```

Output:

```
Matrice:
  1   2   3   4
  5   6   7   8
  9  10  11  12
```

---

4. Operazioni su Matrici

Input da utente:

```c
#include <stdio.h>

#define RIGHE 3
#define COLONNE 3

int main() {
    int matrice[RIGHE][COLONNE];

    printf("Inserisci gli elementi della matrice %dx%d:\n", RIGHE, COLONNE

    for (int i = 0; i < RIGHE; i++) {
        for (int j = 0; j < COLONNE; j++) {
            printf("Elemento [%d][%d]: ", i, j);
            scanf("%d", &matrice[i][j]);
        }
    }

    return 0;
}
```

Somma di matrici:

```c
// C = A + B
for (int i = 0; i < righe; i++) {
    for (int j = 0; j < colonne; j++) {
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

Trasposizione (scambio righe ↔ colonne):

```c
// Matrice originale: 3×4
// Matrice trasposta: 4×3

for (int i = 0; i < righe; i++) {
    for (int j = 0; j < colonne; j++) {
        trasposta[j][i] = originale[i][j];
    }
}
```

Prodotto matrice per scalare:

```c
int scalare = 5;

for (int i = 0; i < righe; i++) {
    for (int j = 0; j < colonne; j++) {
        risultato[i][j] = matrice[i][j] * scalare;
    }
}
```

Ricerca elemento:

```c
int target = 42;
int trovato = 0;
int posizioneI, posizioneJ;

for (int i = 0; i < righe; i++) {
    for (int j = 0; j < colonne; j++) {
        if (matrice[i][j] == target) {
            trovato = 1;
            posizioneI = i;
            posizioneJ = j;
            break;
        }
    }
    if (trovato) break;
}
```

---

5. Stringhe in C: Array di Caratteri

In C, una **stringa** è un array di caratteri terminato dal carattere speciale **'\0'** (null terminator).

Rappresentazione in memoria:

```
Stringa: char nome[] = "Ciao";

Indice:    0    1    2    3    4
          ┌────┬────┬────┬────┬────┐
nome      │ 'C'│ 'i'│ 'a'│ 'o'│ '\0'│
          └────┴────┴────┴────┴────┘
```

Dichiarazione e inizializzazione:

```c
// Metodo 1: Inizializzazione diretta con stringa letterale
char nome[] = "Mario";        // Dimensione automatica = 6 (5 + '\0')

// Metodo 2: Specificare dimensione
char cognome[20] = "Rossi";   // cognome[5] = '\0', resto = spazio inutili

// Metodo 3: Carattere per carattere
char saluto[6] = {'C', 'i', 'a', 'o', '!', '\0'};  // '\0' OBBLIGATORIO!

// Metodo 4: Senza inizializzazione
char buffer[100];  // Contiene valori casuali

// ERRORE COMUNE: Dimenticare '\0'
char sbagliato[4] = {'C', 'i', 'a', 'o'};  // ERRORE! Manca '\0'
```

Importante: - Il terminatore '\0' è **obbligatorio** - La dimensione dell'array deve essere **lunghezza + 1** - Le stringhe letterali (tra virgolette) includono automaticamente '\0'

---

6. Terminatore '\0'

Il carattere **'\0'** (valore ASCII = 0) indica la fine della stringa.

Perché è necessario?

```c
char nome[10] = "Anna";

// In memoria:
// [0]='A', [1]='n', [2]='n', [3]='a', [4]='\0', [5-9]=garbage

// Le funzioni sanno dove fermarsi grazie a '\0'
printf("%s", nome);  // Stampa "Anna" e si ferma a '\0'
```

Cosa succede senza '\0'?

```c
char malformata[4] = {'T', 'e', 's', 't'};  // Manca '\0'!

printf("%s", malformata);
// Stampa "Test" + caratteri casuali finché non trova un '\0'
// COMPORTAMENTO INDEFINITO!
```

Aggiungere manualmente '\0':

```c
char parola[10];
parola[0] = 'C';
parola[1] = 'i';
parola[2] = 'a';
parola[3] = 'o';
parola[4] = '\0';  // FONDAMENTALE!

printf("%s\n", parola);  // Output: Ciao
```

---

7. Input/Output di Stringhe

Output con printf():

```
char nome[] = "Mario";

printf("%s\n", nome);           // Stampa "Mario"
printf("Nome: %s\n", nome);     // Stampa "Nome: Mario"
printf("%10s\n", nome);         // Stampa "     Mario" (allineato a destra
printf("%-10s\n", nome);        // Stampa "Mario     " (allineato a sinist
```

Input con scanf():

```
char nome[50];

printf("Inserisci il tuo nome: ");
scanf("%s", nome);  // ATTENZIONE: legge fino al primo spazio!

// Problema: "Mario Rossi" → legge solo "Mario"
```

Input con gets() (DEPRECATO - NON USARE!):

```
char nome[50];
gets(nome);  // PERICOLOSO! Non controlla la dimensione
```

Input con fgets() (RACCOMANDATO):

```
char nome[50];

printf("Inserisci il tuo nome completo: ");
fgets(nome, sizeof(nome), stdin);  // Sicuro: limita a 49 caratteri + '\0'

// fgets() include '\n' alla fine, potrebbe servire rimuoverlo
nome[strcspn(nome, "\n")] = '\0';
```

---

8. Introduzione alle Funzioni string.h

La libreria **string.h** fornisce funzioni per manipolare stringhe.

Funzioni principali:

```c
#include <string.h>

// strlen() - Lunghezza della stringa (senza '\0')
char parola[] = "Ciao";
int lunghezza = strlen(parola);  // lunghezza = 4

// strcpy() - Copia una stringa
char dest[20];
char src[] = "Hello";
strcpy(dest, src);  // dest = "Hello"

// strcat() - Concatena stringhe
char str1[20] = "Hello";
char str2[] = " World";
strcat(str1, str2);  // str1 = "Hello World"

// strcmp() - Confronta stringhe
char s1[] = "abc";
char s2[] = "abc";
char s3[] = "xyz";

if (strcmp(s1, s2) == 0) {
    printf("s1 e s2 sono uguali\n");
}
// strcmp() ritorna:
//   0  se uguali
//   <0 se s1 < s2
//   >0 se s1 > s2

// strchr() - Cerca carattere
char str[] = "Hello World";
char* pos = strchr(str, 'W');  // pos punta a "World"

// strstr() - Cerca sottostringa
char* pos2 = strstr(str, "World");  // pos2 punta a "World"
```

ATTENZIONE alla sicurezza:

```c
// VERSIONI SICURE (con limite di dimensione)
strncpy(dest, src, n);   // Copia max n caratteri
strncat(dest, src, n);   // Concatena max n caratteri
strncmp(s1, s2, n);      // Confronta max n caratteri
```

PARTE PRATICA (2h)

Esercizio 1: Operazioni su Matrici

```c
#include <stdio.h>

#define RIGHE 3
#define COLONNE 3

void stampaMatrice(int mat[][COLONNE], int r, int c) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            printf("%4d ", mat[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

int main() {
    int matrice[RIGHE][COLONNE];

    // Input matrice
    printf("Inserisci gli elementi della matrice %dx%d:\n", RIGHE, COLONNE
```

```c
for (int i = 0; i < RIGHE; i++) {
    for (int j = 0; j < COLONNE; j++) {
        printf("Elemento [%d][%d]: ", i, j);
        scanf("%d", &matrice[i][j]);
    }
}

printf("\nMatrice inserita:\n");
stampaMatrice(matrice, RIGHE, COLONNE);

// Somma di tutti gli elementi
int somma = 0;
for (int i = 0; i < RIGHE; i++) {
    for (int j = 0; j < COLONNE; j++) {
        somma += matrice[i][j];
    }
}
printf("Somma totale: %d\n\n", somma);

// Somma per righe
printf("Somma per righe:\n");
for (int i = 0; i < RIGHE; i++) {
    int sommaRiga = 0;
    for (int j = 0; j < COLONNE; j++) {
        sommaRiga += matrice[i][j];
    }
    printf("Riga %d: %d\n", i, sommaRiga);
}
printf("\n");

// Somma per colonne
printf("Somma per colonne:\n");
for (int j = 0; j < COLONNE; j++) {
    int sommaColonna = 0;
    for (int i = 0; i < RIGHE; i++) {
        sommaColonna += matrice[i][j];
    }
    printf("Colonna %d: %d\n", j, sommaColonna);
}
printf("\n");

// Diagonale principale
printf("Diagonale principale: ");
for (int i = 0; i < RIGHE; i++) {
    printf("%d ", matrice[i][i]);
}
printf("\n");

// Diagonale secondaria
printf("Diagonale secondaria: ");
for (int i = 0; i < RIGHE; i++) {
    printf("%d ", matrice[i][COLONNE - 1 - i]);
}
printf("\n\n");

// Elemento massimo e minimo
int max = matrice[0][0], min = matrice[0][0];
int maxI = 0, maxJ = 0, minI = 0, minJ = 0;

for (int i = 0; i < RIGHE; i++) {
    for (int j = 0; j < COLONNE; j++) {
        if (matrice[i][j] > max) {
            max = matrice[i][j];
            maxI = i;
            maxJ = j;
        }
        if (matrice[i][j] < min) {
            min = matrice[i][j];
            minI = i;
            minJ = j;
```

```
                minJ = j;
            }
        }
    }

    printf("Massimo: %d in posizione [%d][%d]\n", max, maxI, maxJ);
    printf("Minimo: %d in posizione [%d][%d]\n", min, minI, minJ);

    return 0;
}
```

Esercizio 2: Trasposizione di Matrice

```c
#include <stdio.h>

#define RIGHE 3
#define COLONNE 4

void stampaMatrice(int mat[][COLONNE], int r, int c) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            printf("%4d ", mat[i][j]);
        }
        printf("\n");
    }
}

void stampaMatriceTr(int mat[][RIGHE], int r, int c) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            printf("%4d ", mat[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int matrice[RIGHE][COLONNE] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    int trasposta[COLONNE][RIGHE];

    printf("Matrice originale (%dx%d):\n", RIGHE, COLONNE);
    stampaMatrice(matrice, RIGHE, COLONNE);

    // Trasposizione
    for (int i = 0; i < RIGHE; i++) {
        for (int j = 0; j < COLONNE; j++) {
            trasposta[j][i] = matrice[i][j];
        }
    }

    printf("\nMatrice trasposta (%dx%d):\n", COLONNE, RIGHE);
    stampaMatriceTr(trasposta, COLONNE, RIGHE);

    // Verifica trasposizione
    printf("\nVerifica: trasposta[1][2] = matrice[2][1]\n");
    printf("trasposta[1][2] = %d, matrice[2][1] = %d\n",
           trasposta[1][2], matrice[2][1]);

    return 0;
}
```

Esercizio 3: Somma di Matrici

```c
#include <stdio.h>

#define DIM 3

void inputMatrice(int mat[][DIM], int dim, const char* nome) {
    printf("Inserisci elementi di %s:\n", nome);
    for (int i = 0; i < dim; i++) {
        for (int j = 0; j < dim; j++) {
            printf("%s[%d][%d]: ", nome, i, j);
            scanf("%d", &mat[i][j]);
        }
    }
}

void stampaMatrice(int mat[][DIM], int dim, const char* nome) {
    printf("\n%s:\n", nome);
    for (int i = 0; i < dim; i++) {
        for (int j = 0; j < dim; j++) {
            printf("%4d ", mat[i][j]);
        }
        printf("\n");
    }
}

void sommaMatrici(int A[][DIM], int B[][DIM], int C[][DIM], int dim) {
    for (int i = 0; i < dim; i++) {
        for (int j = 0; j < dim; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }
}

int main() {
    int A[DIM][DIM], B[DIM][DIM], C[DIM][DIM];

    inputMatrice(A, DIM, "Matrice A");
    inputMatrice(B, DIM, "Matrice B");

    sommaMatrici(A, B, C, DIM);

    stampaMatrice(A, DIM, "Matrice A");
    stampaMatrice(B, DIM, "Matrice B");
    stampaMatrice(C, DIM, "Matrice C (A+B)");

    return 0;
}
```

Esercizio 4: Manipolazione di Stringhe Base

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100], str3[200];

    // Input stringhe
    printf("Inserisci la prima stringa: ");
    fgets(str1, sizeof(str1), stdin);
    str1[strcspn(str1, "\n")] = '\0';  // Rimuove '\n'

    printf("Inserisci la seconda stringa: ");
    fgets(str2, sizeof(str2), stdin);
    str2[strcspn(str2, "\n")] = '\0';

    // Lunghezza
    printf("\nLunghezza str1: %lu caratteri\n", strlen(str1));
    printf("Lunghezza str2: %lu caratteri\n", strlen(str2));

    // Copia
    strcpy(str3, str1);
    printf("\nstr3 (copia di str1): %s\n", str3);

    // Concatenazione
    strcat(str3, " ");
    strcat(str3, str2);
    printf("str3 (concatenazione): %s\n", str3);

    // Confronto
    printf("\nConfronto str1 e str2:\n");
    int risultato = strcmp(str1, str2);
    if (risultato == 0) {
        printf("Le stringhe sono uguali\n");
    } else if (risultato < 0) {
        printf("str1 < str2 (lessicograficamente)\n");
    } else {
        printf("str1 > str2 (lessicograficamente)\n");
    }

    // Ricerca carattere
    printf("\nInserisci un carattere da cercare in str1: ");
    char ch;
    scanf(" %c", &ch);

    char* pos = strchr(str1, ch);
    if (pos != NULL) {
        printf("Carattere '%c' trovato in posizione %ld\n", ch, pos - str1
    } else {
        printf("Carattere '%c' non trovato\n", ch);
    }

    return 0;
}
```

Esercizio 5: Conteggio Caratteri e Parole

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char testo[1000];

    printf("Inserisci un testo (max 999 caratteri):\n");
    fgets(testo, sizeof(testo), stdin);
    testo[strcspn(testo, "\n")] = '\0';
```

```c
// Statistiche
int lunghezza = strlen(testo);
int lettere = 0, cifre = 0, spazi = 0, punteggiatura = 0, altri = 0;
int vocali = 0, consonanti = 0;
int parole = 0;

// Conteggi
int inParola = 0;
for (int i = 0; i < lunghezza; i++) {
    char c = testo[i];

    // Conta parole
    if (isspace(c)) {
        spazi++;
        inParola = 0;
    } else {
        if (!inParola) {
            parole++;
            inParola = 1;
        }
    }

    // Classifica caratteri
    if (isalpha(c)) {
        lettere++;
        char lower = tolower(c);
        if (lower == 'a' || lower == 'e' || lower == 'i' ||
            lower == 'o' || lower == 'u') {
            vocali++;
        } else {
            consonanti++;
        }
    } else if (isdigit(c)) {
        cifre++;
    } else if (ispunct(c)) {
        punteggiatura++;
    } else if (!isspace(c)) {
        altri++;
    }
}

// Output statistiche
printf("\n=== STATISTICHE ===\n");
printf("Lunghezza totale: %d caratteri\n", lunghezza);
printf("Numero di parole: %d\n", parole);
printf("\nCaratteri:\n");
printf("  Lettere: %d (vocali: %d, consonanti: %d)\n", lettere, vocali
printf("  Cifre: %d\n", cifre);
printf("  Spazi: %d\n", spazi);
printf("  Punteggiatura: %d\n", punteggiatura);
printf("  Altri: %d\n", altri);

// Frequenza lettere
int frequenza[26] = {0};
for (int i = 0; i < lunghezza; i++) {
    if (isalpha(testo[i])) {
        frequenza[tolower(testo[i]) - 'a']++;
    }
}

printf("\nFrequenza lettere:\n");
for (int i = 0; i < 26; i++) {
    if (frequenza[i] > 0) {
        printf("%c: %d ", 'a' + i, frequenza[i]);
    }
}
printf("\n");

return 0;
```

```
}
```

Esercizi da svolgere autonomamente:

1. **Matrice identità**: Crea e verifica se una matrice è una matrice identità
2. **Prodotto matrici**: Implementa il prodotto tra due matrici compatibili
3. **Matrice simmetrica**: Verifica se una matrice è simmetrica rispetto alla diagonale principale
4. **Palindromo**: Verifica se una stringa è palindroma (si legge uguale al contrario)
5. **Inversione stringa**: Inverti i caratteri di una stringa senza usare funzioni di libreria
6. **Conta occorrenze**: Data una stringa e una sottostringa, conta quante volte appare
7. **Rimozione spazi**: Rimuovi tutti gli spazi da una stringa
8. **Cifrario di Cesare**: Implementa il cifrario di Cesare (shift di caratteri)
9. **Anagrammi**: Verifica se due stringhe sono anagrammi l'una dell'altra
10. **Serpente in matrice**: Riempi una matrice NxN con numeri da 1 a N^2 a serpente (zig-zag)

Riepilogo Concetti Chiave

✓ Le **matrici** sono array bidimensionali: `tipo[righe][colonne]`
✓ Accesso: `matrice[i][j]` con cicli annidati
✓ Le **stringhe** sono array di char terminati da **'\0'**
✓ Funzioni string.h: strlen, strcpy, strcat, strcmp
✓ Usare **fgets()** per input sicuro di stringhe
✓ Il terminatore '\0' è fondamentale per le stringhe

Prossima lezione: Approfondiremo le **funzioni** in C - parte 1!